

APPENDIX

In this appendix, we provide additional information to supplement the design and implementation of our framework, our experiment settings, and additional results. Appendix A discusses additional implementation details, including a detailed description of our architecture, further graph-based communication protocols, description of the utilized libraries, and hyperparameter settings. In Appendix B, we outline the algorithm used for centralized meta-training and decentralized meta-testing of GCD. Moreover, Appendix C describes additional details on the experiment domain and datasets used in our experiments. Finally, in Appendix D, we provide additional experimental results.

A IMPLEMENTATION DETAILS

A.1 Detailed Architecture

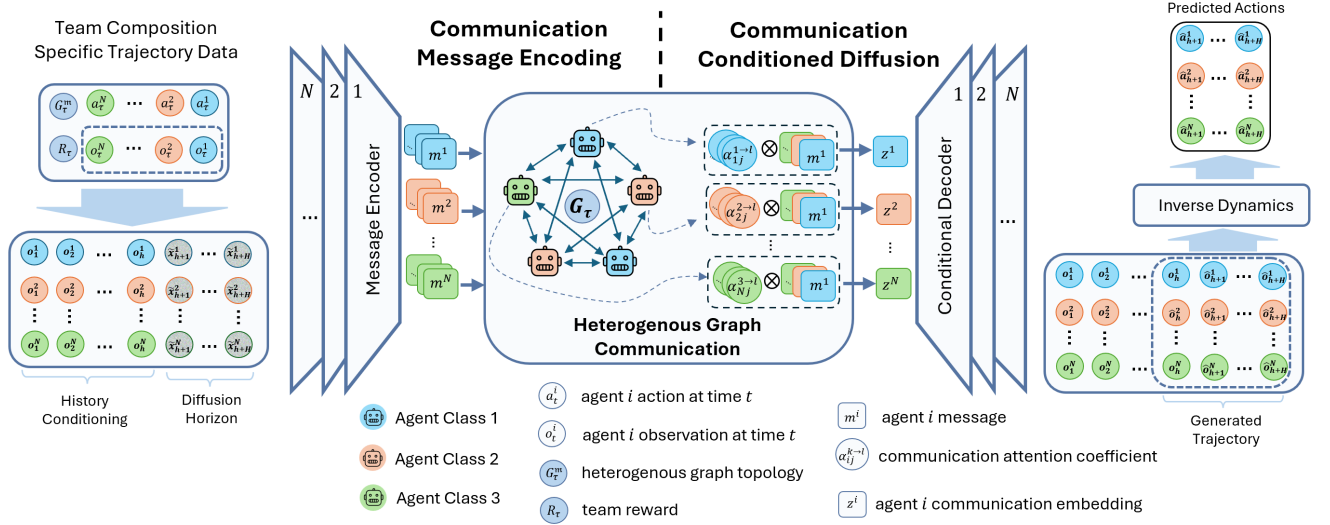


Figure 4: Overview of GCD-HetGAT: Team-specific observations, actions, rewards, and the heterogeneous graph topology are used to encode information as messages within a multi-agent communication framework. Agents use HetGAT to generate communication embeddings, conditioning the diffusion model’s trajectory generation process.

GCD’s diffusion architecture builds on the convolutional U-Net architecture modified for trajectory generation [5] and the attentional-mechanisms used for multi-agent coordination in [21]. We depict our architecture in Figure 1 of Section 4, with Figure 4 showing the GCD-HetGAT variant. Each convolutional block consists of a 1D convolutional layer, followed by a group normalization layer [18] and Mish nonlinearity [11]. Additionally, the diffusion timestep, k , and the trajectory reward conditioning are embedded through fully-connected layers in the first convolutional block.

GCD introduces two key advancements. First, we advance beyond the standard homogeneous transformer-based attention mechanism used by Zhu et al. [21] through the integration of graph-based methods for modeling multi-agent communication. This integration allows us to effectively leverage the underlying task structure encoded in the communication graph, G_r^m , across diverse team compositions. Moreover, by modeling agents as nodes in a graph, this enables decentralized execution without the need for explicit teammate and opponent modeling. In Figure 1, we depict how during the diffusion process, each agent i processes its own observations through a series of convolutional layers, generating its own message embedding, m_r^{ij} , at each layer r . Through a message passing mechanism (which can be implemented in a decentralized fashion), each agent accumulates its own message inbox, m_r^{-i} , where $-i$ indicates the agent index of all other agents. Each agent, equipped with distinct graph-based communication protocols at each layer, processes its own message, its message inbox, and the graph structure G_r^m , where node-types represent each agent’s class and edge-types represent class-to-class communication channels. Each agent then computes its communication embedding z_r^{ij} using the corresponding protocol (GCN, HetGCN, and HetGAT).

Our second advancement involves modifying the learning process to learn conditional data distributions during training, conditioned not only on the team’s returns but also on the team compositions. We enable this in our architecture by utilizing the embeddings derived from multi-agent communication as the conditioning signal, as it provides a suitable basis of task-relevant information. We train both conditional and unconditional models simultaneously by applying random dropout sampling to the communication embeddings, with the dropout probability determined by the Bernoulli random variable β_z , a standard technique in classifier-free diffusion models [4]. We depict this in our architecture in Figure 1 by adding the label $(1 - \beta_z)z_r^{ij}$ to the connections from the graph-based communication layers to the up-convolutional layers.

A.2 Graph-Based Communication Mechanisms

In Section 4.1.2, we describe how task-relevant embeddings are computed using a HetGAT communication protocol that leverages the heterogeneous graph structure. This mechanism is employed in our GCD-HetGAT models. Additionally, in this section, we describe alternative graph-based communication mechanisms used in the GCD-Homo and GCD-HetGCN models.

GCD-Homo employs a Graph Convolutional Network (GCN) [6] as the graph-based communication protocol. The computation of the communication embeddings is described in Eq. 5. Notably, the communication embeddings computed by GCD-Homo are processed through homogeneous layers and do not account for type-specific or relation-specific information.

$$z_r^i = \sigma\left(Wm_r^i + \sum_{k \in N(i)} Wm_r^k\right) \quad (5)$$

GCD-HetGCN employs a heterogeneous GCN [14] as the graph-based communication protocol. The computation of the communication embeddings is described in Eq. 6. Here, the communication protocol applies type-specific layers for an agent’s own message, and relation-specific layers for messages pass from neighboring agent types of class l to j . This mechanism allows the communication protocol to encode the heterogeneous structure of team compositions into the representations used as the conditioning signal. We note that this communication protocol can be viewed as a simplified version of HetGAT communication, omitting additional layers used to compute attention coefficients during message aggregation.

$$z_r^{ij} = \sigma\left(W_j m_r^{ij} + \sum_{l \in C} \sum_{k \in N_l(i)} W_{l \rightarrow j} m_r^{kl}\right) \quad (6)$$

A.3 Implementation Frameworks and Libraries

Our implementation of GCD is based on the publicly available implementation of MADiff [21], building the on the shared multi-agent U-Network models and diffusion training processes. We extend these implementations to integrate graph-based multi-agent communication during the diffusion process. We use PyTorch [13] and the Deep Graph Library (DGL) [17], popular frameworks for deep learning with Graph Neural Networks (GNNs), leveraging CUDA-accelerated batched operations for efficient graph-based message passing and aggregation during training and evaluation. Our implementation of GCD-Homo utilizes the standard Graph-Convolutional Layer modules provided by DGL, while GCD-HetGCN utilizes the standard Relational Graph-Convolutional Layer modules. For GCD-HetGAT, we implement our module utilizing the standard graph operation utilities provided by DGL. For training offline MARL baselines, we use the codebase of MA-CQL [15], MA-ICQ [19], MA-IQL [7], OMAR [12], MASIA [2], and MHCI [10], respectively. We utilize the author-provided codebase of HiSSD [9] for running both ODIS and HiSSD. Our dataset and codebase, along with the implementations of these baselines, are available at: [TBD](#).

A.4 Hyperparameters

We summarize all hyperparameters used for running experiments with GCD in Table 2. Since GCD builds on the MADiff baseline, all shared hyperparameters are also applied in MADiff experiments.

Hyperparameter	Value	Description
α	2e-4	Learning rate
h	20	History conditioning horizon
H	4	Diffusion horizon
R	3	Number of communication layers
T_r	[24, 12, 6]	Agent message dimensionality along the time-horizon at layer r
d_r	[128, 512, 1024]	Agent message dimensionality along the feature dimension at layer r
β_r	0.1	Dropout probability for the trajectory reward embedding
β_z	0.1	Dropout probability for the communication embedding
ω	1.2	Classifier-free guidance weight
K_{train}	200	Number of diffusion timesteps used with DDPM [3] for training
K_{sample}	15	Number of diffusion timesteps used with DDIM [16]
B	32	Batch size
N_{train}	500,000	Number of training timesteps

Table 2: Hyperparameter settings used in experiments.

B ALGORITHM

Algorithm 1: Centralized meta-training of Graph Conditioned Diffusion using HetGAT communication embeddings

Input: Noise model ϵ_θ , Inverse dynamics f_ϕ , offline dataset $\mathcal{D}$

```

1 while not done do
2   Sample  $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ ,  $k \sim \mathcal{U}(1, K)$ ,  $\beta \sim \text{Bern}(p)$ 
3   Sample  $d_m = [\mathcal{G}_\tau^m, \mathcal{R}_\tau^m, \bar{\mathbf{a}}_\tau^m, \bar{\mathbf{o}}_\tau^m] \sim \mathcal{D}_m \subset \mathcal{D}$ 
4    $\bar{\mathbf{x}}_k^m \leftarrow [\bar{\mathbf{o}}_{1:h}^m || [\bar{\mathbf{x}}_{k,h+1:h+H}^m]$  with  $\bar{\mathbf{x}}_{k,t}^m = \sqrt{\bar{\alpha}^k} \mathbf{x}_0^m + \sqrt{1 - \bar{\alpha}^k} \epsilon$  ▷ Initialize joint diffusion state with history conditioning
5    $\bar{\mathbf{m}}_\tau^m = \text{MessageEncoder}(\bar{\mathbf{x}}_k^m)$  ▷ Compute agent messages
6   CentralizedMessagePassing( $\bar{\mathbf{m}}_\tau^m, \mathcal{G}_\tau^m$ )
7    $\bar{\mathbf{z}}_\tau^m \sim \text{HetGAT}(\bar{\mathbf{m}}_\tau^m, \mathcal{G}_\tau^m)$ 
8   Compute  $\mathcal{L}_{\theta,\phi}^T$  according to Eq 3-4
9    $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}_{\theta,\phi}^T$  ▷ Update parameters of diffusion model
10   $\phi \leftarrow \phi - \alpha \nabla_\phi \mathcal{L}_{\theta,\phi}^T$  ▷ Update parameters of the inverse dynamics model

```

Algorithm 2: Decentralized meta-testing with graph Conditioned Diffusion using HetGAT communication embeddings

Input: Noise model ϵ_θ , Inverse dynamics f_ϕ , history h , horizon H , heterogeneous graph $\mathcal{G}_\tau^m$, guidance scale ω

```

1 Initialize time with  $t \leftarrow 1$ 
2 while not done do
3   Initialize  $\mathbf{x}_K^{ij} = [\mathbf{o}_{1:h}^{ij} || \bar{\mathbf{x}}_{K,h+1:h+H}^{ij}]$  with  $\bar{\mathbf{x}}_{K,t}^{ij} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
4   for  $k=K \dots 1$  do
5     foreach agent  $i$  of class  $j$  do
6        $\mathbf{m}_\tau^{ij} = \text{MessageEncoder}(\mathbf{x}_k^{ij})$ 
7        $\bar{\mathbf{m}}_\tau \leftarrow \text{DecentralizedMessagePassing}(\mathbf{m}_\tau^{ij}, \mathcal{G}_\tau^m)$ 
8        $\bar{\mathbf{z}}_\tau^{ij} = \text{HetGAT}(\bar{\mathbf{m}}_\tau)$ 
9        $\hat{\epsilon} := \epsilon_\theta(\mathbf{x}_k^{ij}, \emptyset, k) + \omega (\epsilon_\theta(\mathbf{x}_k^{ij}, (\bar{\mathbf{z}}_\tau^{ij}, R_\tau), k) - \epsilon_\theta(\mathbf{x}_k^{ij}, \emptyset, k))$  ▷ Communication Conditioned Diffusion
10       $(\mu_{k-1}^{ij}, \alpha \Sigma_{k-1}^{ij}) \leftarrow \text{Denoise}(\mathbf{x}_k^{ij}, \hat{\epsilon})$ 
11       $\mathbf{x}_k^{ij} \sim \mathcal{N}(\mu_{k-1}^{ij}, \alpha \Sigma_{k-1}^{ij})$ 
12    foreach agent  $i$  of class  $j$  do
13       $(\mathbf{o}_t^{ij}, \mathbf{o}_{t+1}^{ij}) \leftarrow \mathbf{x}_0^{ij} [t : t+1]$ 
14      Compute  $\hat{\mathbf{a}}_t^{ij} = f_\phi(\mathbf{o}_t^{ij}, \mathbf{o}_{t+1}^{ij})$ 
15      Execute action in the environment  $t \leftarrow t+1$ 
16      Observe  $\mathbf{o}_{t+1}^{ij}$  and update history  $\mathbf{o}_{1:h}^{ij}$ 

```

C DOMAIN AND DATASET DETAILS

C.1 Domain

Our experiments are conducted on the SMACv2 [1] domain, where we evaluate the performance of our methods across various team compositions and scenarios to assess their ability to coordinate effectively and achieve successful outcomes. Our results and discussion in Section 6 correspond to experiments on the terran_5_vs_5 and protoss_5_vs_5. Moreover, additional results presented in Section D are conducted on the zerg_5_vs_5 scenario. In the terran_5_vs_5 scenario, the available unit classes are Marine (MA), Marauder (MR), and Medivac (MD). In the protoss_5_vs_5 scenario, the unit classes include Stalker (ST), Zealot (ZE), and Colossus (CO). In the zerg_5_vs_5 scenario, the unit classes are Zergling (Z), Baneling (B), and Hydralisk (H).

As described in Section 5.4, during evaluation every episode contains the same team composition, in contrast to the standard SMACv2 scenarios where team compositions are randomly sampled with different weights for different unit types. This approach allows us to focus our analysis on how effectively our algorithm can condition and adapt to each specific team composition.

C.2 Datasets

terrann 5 vs 5				protoss 5 vs 5				zerg 5 vs 5			
Training tasks	Time steps	Win Rate	# of Ep.	Training tasks	Time steps	Win Rate	# of Ep.	Training tasks	Time steps	Win Rate	# of Ep.
2ma_3md	67033	0.800	600	2st_3co	41550	0.875	600	2h_3b	15155	0.800	600
2ma_3mr	23158	0.756	600	2st_3ze	36936	0.475	600	2z_3b	8400	0.975	600
2mr_3md	64338	0.575	600	2ze_3co	40171	0.900	600	2z_3h	19466	0.625	600
3ma_2md	44161	0.725	600	3st_2co	39566	0.829	600	3h_2b	16374	0.600	600
3ma_2mr	22581	0.805	600	3st_2ze	37376	0.525	600	3z_2b	9254	1.00	600
3mr_2md	32899	0.700	600	3ze_2co	36345	0.975	600	3z_2h	19013	0.775	600

Table 3: Dataset setup for SMACv2 scenarios.

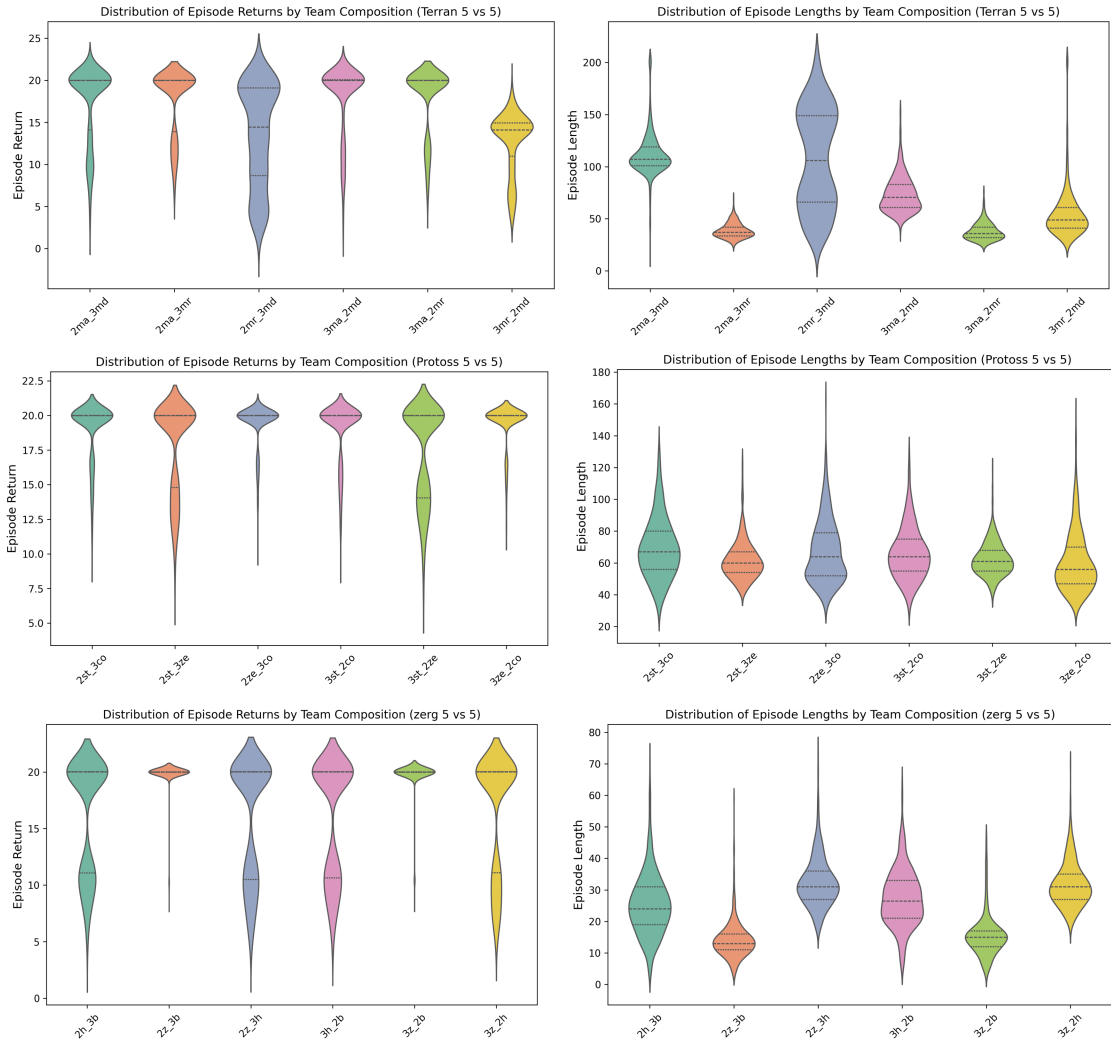


Figure 5: Violin plots showing the distributions of episode returns (left) and episode lengths (right) across different team compositions for the terrann_5_vs_5 (top), protoss_5_vs_5 (middle), and zerg_5_vs_5 (bottom) scenarios.

Here, we provide details about our training dataset. As discussed in Section 5.2, we employed HATRPO [8] as expert policies and trained distinct policies for each team composition in the set of training tasks. This design choice allows us to capture coordination behaviors specific to each team structure. For implementation, we used the official HARL codebase [20] without any modifications, except that we applied fixed team compositions to train policies tailored to specific teams. Note that, as our method utilizes a graph structure data, we further augment each sample with $\mathcal{G}_r^m$, encoding team composition-specific information as described in Section 4.1.1.

Table 3 and Figure 5 summarize the details of our expert dataset. We selected 6 out of the 20 possible team compositions for the terran_5_vs_5, protoss_5_vs_5, and zerg_5_vs_5 scenarios. Each chosen team composition consists of exactly two unit types, arranged in a fixed 2-3 or 3-2 split (e.g., 2ma_3md, 3ma_2mr). In the terran_5_vs_5 scenario, longer episodes are observed for 2ma_3md and 2mr_3md, as noted in Section 6.1.1. This is because Medivac units serve a supportive role by healing allies rather than attacking enemies, resulting in prolonged scenarios and ultimately lower win rates. Regarding the protoss_5_vs_5 scenario, we observe that when Colossus—a highly powerful unit—is included, the expert policy achieves higher win rates compared to other compositions such as 2st_3ze and 3st_2ze. Similarly, in the zerg_5_vs_5 scenario, the Baneling is a self-explosive unit with high burst damage. Its presence produces an effect comparable to that of the Colossus in terms of impact; however, it leads to shorter episode durations compared to the other two scenarios due to its high-risk, high-reward nature. These observations suggest that inter-agent cooperation may play a less critical role in protoss_5_vs_5 and zerg_5_vs_5 than in terran_5_vs_5, where the absence of a single overwhelmingly dominant unit places greater emphasis on coordinated team dynamics.

D ADDITIONAL RESULTS

D.1 Algorithm Analysis

In Figure 6, we provide full results on the performance improvements of GCD relative to the baselines.

D.2 Ablation Study

As shown in Figure 7, we present the results of ablation experiments for the terran_5_vs_5, protoss_5_vs_5, and zerg_5_vs_5 scenarios. Our ablation variants correspond to GCD-Homo, GCD-HetGCN, and GCD-HetGAT.

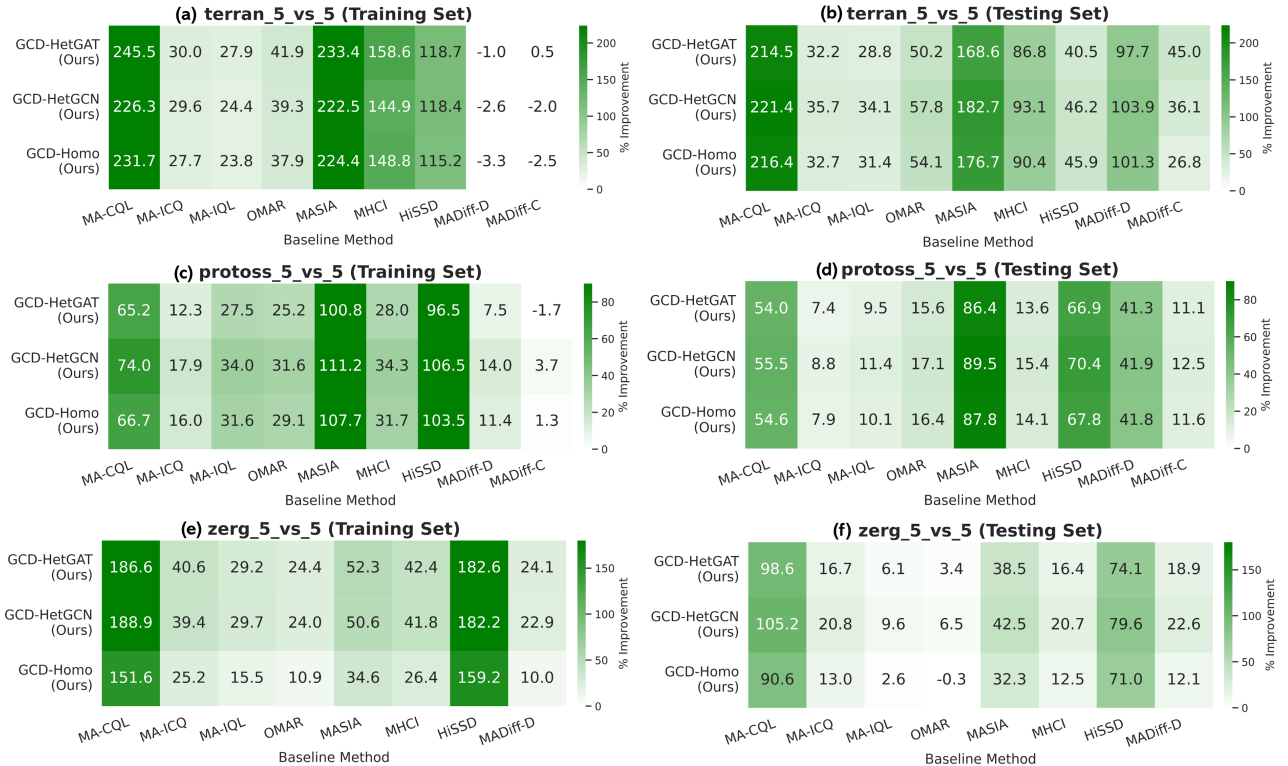


Figure 6: GCD’s improvement over baselines for the training (left) and testing (right) sets across the terran_5_vs_5 (top), protoss_5_vs_5 (middle), and zerg_5_vs_5 (bottom) scenarios.

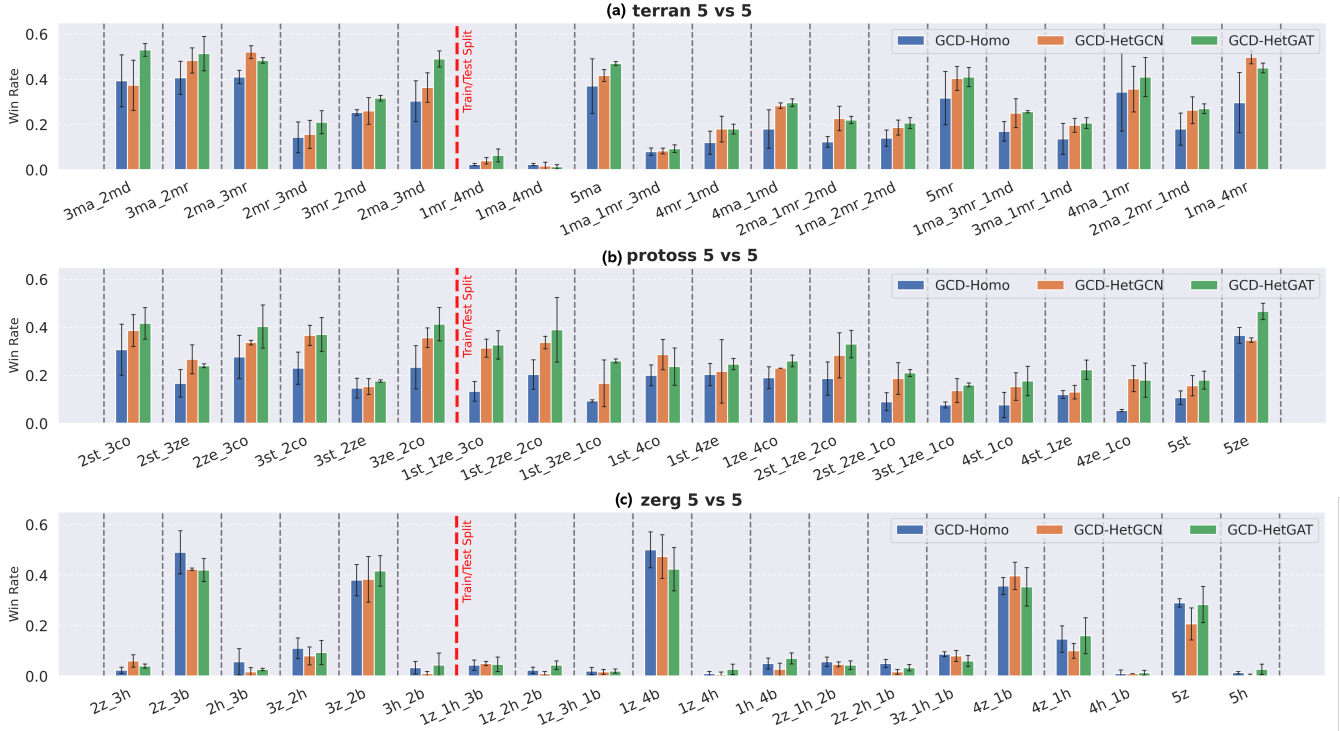


Figure 7: Results of ablation experiments for terran_5_vs_5 (top), protoss_5_vs_5 (bottom), and zerg_5_vs_5 scenarios. We show the average win-rates ($\pm$ standard deviation) across GCD-Homo, GCD-HetGCN, and GCD-HetGAT, grouped by team compositions labels in the x-axis. The dashed red vertical line indicates the boundary between team compositions used for training (left) and unseen test compositions (right).

REFERENCES

- [1] Benjamin Ellis, Jonathan Cook, Skander Moalla, Mikayel Samvelyan, Mingfei Sun, Anuj Mahajan, Jakob Foerster, and Shimon Whiteson. 2024. Smacv2: An improved benchmark for cooperative multi-agent reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2024).
- [2] Cong Guan, Feng Chen, Lei Yuan, Zongzhang Zhang, and Yang Yu. 2024. Efficient Communication via Self-Supervised Information Aggregation for Online and Offline Multiagent Reinforcement Learning. *IEEE Transactions on Neural Networks and Learning Systems* (2024).
- [3] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. *Advances in neural information processing systems* 33 (2020), 6840–6851.
- [4] Jonathan Ho and Tim Salimans. 2022. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598* (2022).
- [5] Michael Janner, Yilun Du, Joshua B Tenenbaum, and Sergey Levine. 2022. Planning with diffusion for flexible behavior synthesis. *arXiv preprint arXiv:2205.09991* (2022).
- [6] TN Kipf. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [7] Ilya Kostrikov, Rob Fergus, Jonathan Tompson, and Ofir Nachum. 2021. Offline reinforcement learning with fisher divergence critic regularization. In *International Conference on Machine Learning*. PMLR, 5774–5783.
- [8] Jakub Grudzien Kuba, Ruiqing Chen, Muning Wen, Ying Wen, Fanglei Sun, Jun Wang, and Yaodong Yang. 2022. Trust Region Policy Optimisation in Multi-Agent Reinforcement Learning. In *International Conference on Learning Representations*.
- [9] Sicong Liu, Yang Shu, Chenjuan Guo, and Bin Yang. 2025. Learning generalizable skills from offline multi-task data for multi-agent cooperation. *arXiv preprint arXiv:2503.21200* (2025).
- [10] Yihuan Mao, Rui Hu, Lulu Zheng, Jianhao Wang, and Chongjie Zhang. [n.d.]. Offline Communication Learning with Multi-source Datasets. ([n.d.]).
- [11] Diganta Misra. 2019. Mish: A self regularized non-monotonic activation function. *arXiv preprint arXiv:1908.08681* (2019).
- [12] Ling Pan, Longbo Huang, Tengyu Ma, and Huazhe Xu. 2022. Plan better amid conservatism: Offline multi-agent reinforcement learning with actor rectification. In *International conference on machine learning*. PMLR, 17221–17237.
- [13] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [14] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. 2018. Modeling relational data with graph convolutional networks. In *European semantic web conference*. Springer, 593–607.
- [15] Jianzhun Shao, Yun Qu, Chen Chen, Hongchang Zhang, and Xiangyang Ji. 2023. Counterfactual conservative q learning for offline multi-agent reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2023), 77290–77312.
- [16] Jiaming Song, Chenlin Meng, and Stefano Ermon. 2020. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502* (2020).
- [17] Minjie Yu Wang. 2019. Deep graph library: Towards efficient and scalable deep learning on graphs. In *ICLR workshop on representation learning on graphs and manifolds*.
- [18] Yuxin Wu and Kaiming He. 2018. Group normalization. In *Proceedings of the European conference on computer vision (ECCV)*. 3–19.
- [19] Yiqin Yang, Xiaoteng Ma, Chenghao Li, Zewu Zheng, Qiyuan Zhang, Gao Huang, Jun Yang, and Qianchuan Zhao. 2021. Believe what you see: Implicit constraint approach for offline multi-agent reinforcement learning. *Advances in Neural Information Processing Systems* 34 (2021), 10299–10312.
- [20] Yifan Zhong, Jakub Grudzien Kuba, Xidong Feng, Siyi Hu, Jiaming Ji, and Yaodong Yang. 2024. Heterogeneous-Agent Reinforcement Learning. *Journal of Machine Learning Research* 25, 32 (2024), 1–67. <http://jmlr.org/papers/v25/23-0488.html>
- [21] Zhengbang Zhu, Minghuan Liu, Liyuan Mao, Bingyi Kang, Minkai Xu, Yong Yu, Stefano Ermon, and Weinan Zhang. 2023. Madiff: Offline multi-agent learning with diffusion models. *arXiv preprint arXiv:2305.17330* (2023).